

RECALL AI

Software Development Plan

Version 1.1

Revision History

Date	Version	Description	Author
21/06/2026	1.0	Initial draft - project inception	Thái Nguyễn Tuấn Kiệt
08/07/2026	1.1	Add Sections 1, 2, 3, 4.2, 4.3 - elaboration phase	Thái Nguyễn Tuấn Kiệt

Revision History	1
1. Introduction	1
2. Project Overview	2
2.1 Project Purpose, Scope, and Objectives	2
2.2 Assumptions and Constraints	3
2.3 Project Deliverables	3
3. Project Organization	4
3.1 Organizational Structure	4
3.2 Roles and Responsibilities	5
4. Management Process	5
4.1 Project Estimates	5
4.2 Project Plan	5
4.2.1 Phase and Iteration Plan	5
4.2.2 Releases	8
4.2.3 Project Schedule	8
4.2.4 Project Resourcing	8
4.3 Project Monitoring and Control	9
4.3.1 Reporting	9
4.3.2 Risk Management	10
4.3.3 Configuration Management	10

1. Introduction

This Software Development Plan (SDP) describes the overall planning, organization, and management approach for the Recall AI project, developed by Group 07 (CAGT) as part of the Introduction to Software Engineering course.

Recall AI is a web-based proactive study assistant that helps university students turn raw study materials into a structured, verifiable knowledge-acquisition process. Unlike passive review tools (re-reading, highlighting), Recall AI uses an AI Examiner to conduct multi-turn oral-style quizzes strictly based on the user's uploaded material, and employs a Concept Graph Engine - a Directed Acyclic Graph (DAG) of prerequisite relationships - to automatically detect and remediate root-cause knowledge gaps through a depth-limited reverse BFS trace-back algorithm.

This document covers:

- Section 2: Project Overview - purpose, scope, assumptions, constraints, and deliverables.
- Section 3: Project Organization - team structure, roles, and responsibilities.
- Section 4: Management Process - estimates, phase/iteration plan, schedule, resourcing, reporting, risk management, and configuration management.

This plan is a living document and will be updated at the beginning of each sprint to reflect current team decisions and progress.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

Purpose

Recall AI addresses three linked problems in student self-study:

Passive learning, shallow review - re-reading and highlighting have low retention effectiveness compared to retrieval practice.

No objective self-assessment - students cannot reliably distinguish genuine understanding from mere text familiarity.

Unplanned review - without a structured plan, students cram before deadlines without prioritizing by concept difficulty or remaining time.

The project's goal is to build a working MVP web application that demonstrates the technical feasibility of the trace-back algorithm: when a student answers a concept incorrectly, the system traverses the prerequisite graph to identify and schedule review of the underlying foundational gap - rather than simply repeating the same question.

Scope (MVP - 5 sprints, 10 weeks)

In Scope	Out of Scope
User authentication (email + password)	Google OAuth / third-party SSO
AI Study Planner (text, image and PDF input)	DOCX direct file parsing
Concept Graph Engine (DAG, trace-back BFS)	Full probabilistic model (ALEKS-style)
Focus Session (Pomodoro timer, auto interruption detection)	Mobile app
AI Examiner (multi-turn interview, 3-turn limit/concept)	Voice input (Speech-to-Text)
Dashboard (concept graph colored by mastery score)	Social/collaborative features

Proactive reminders (urgency-score based)

Third-party calendar integration

Objectives

- Deliver a working MVP by Sprint 5 (Aug 23, 2026) that satisfies all 8 primary use cases (UC-01 to UC-08).
- Demonstrate the Concept Graph Engine with real trace-back behavior on at least one end-to-end demo scenario.
- Maintain a stable, testable codebase - the trace-back algorithm must be unit-testable with mock data, independent of AI quality.
- Complete all 5 PA assignments on time and with sufficient quality.

2.2 Assumptions and Constraints

Assumptions:

- All 5 team members are available for the full 10-week project duration (Jun 21 – Aug 23, 2026).
- The Gemini API free tier provides sufficient quota for development and demo use (≤ 50 sessions/day).
- Target users are first- and second-year university students studying subjects with layered concept structures (e.g., algorithms, networks, operating systems).
- Study materials submitted by users are readable plain text or clear photographs; heavily handwritten or low-resolution image inputs are out of scope for MVP.
- The Concept Graph Engine's trace-back quality depends on the AI correctly extracting prerequisite relationships; the mandatory human-in-the-loop confirmation step (UC-03) mitigates incorrect AI extractions.

Constraints:

- Fixed schedule - 10 weeks (5×2 -week sprints). No schedule extension is possible due to course deadlines.
- Zero budget - all tools and services must use free tiers (Gemini API free tier, Vercel/Render free hosting, GitHub free, Figma free).
- Team size fixed at 5 - no additional members may be added.
- C4: AI must not be used as an orchestrator - all routing decisions (when to stop, when to switch concepts, when to trigger trace-back) are deterministic software logic, not AI decisions. AI is called only for generate_question and grade_answer with fixed JSON output schemas.
- C5: The AI Examiner must never generate questions outside the user's uploaded material - no external knowledge fabrication.
- C6: The maximum number of question-answer turns per concept is fixed at 3 turns to control API cost and response time.

2.3 Project Deliverables

A list of the artifacts to be created during the project, including target delivery dates.

Deliverable	Description	Target Date
PA0 - Project Proposal	Project proposal document, team info, interview evidence, tech stack	Jun 28, 2026
PA1 - Requirements Specification	Vision document, user personas, user stories (23 stories across 5 modules), problem statements, hypothesis statements	Jul 12, 2026

PA2 - Use Case Model & SDP	Use case diagram (Draw.io), 8 use case specifications (RUP format), Software Development Plan (this document)	Jul 12, 2026
PA3 - Working Software v1	Auth, Dashboard skeleton, AI Study Planner, Concept Graph Engine with mock data, Focus Session	Jul 26, 2026
PA4 - Working Software v2	AI Examiner (multi-turn state machine), Concept Graph Engine connected to real grading data, test conversation flows	Aug 09, 2026
PA5 - Final Product + Demo	Dashboard concept graph visualization, agentic reminder layer, full system test, final demo	Aug 23, 2026
Weekly Reports	Weekly status reports for every sprint week, tracking completed tasks, blockers, and planned work	Every Tuesday

3. Project Organization

3.1 Organizational Structure

The project team follows a flat, role-based Agile structure with a Project Manager coordinating sprint planning and reporting. There is no external reviewer - all review is done peer-to-peer within the team.

Role	Member	Responsibilities
Project Manager & UI/UX Designer	Thái Nguyễn Tuấn Kiệt	Sprint planning & tracking, JIRA board management, UX research, Figma wireframes & hi-fi mockups, weekly status reports, PA submission coordination
System Architect & Fullstack Dev	Nguyễn Thế Quân	System architecture design, database schema design (PostgreSQL/Prisma, concepts & concept_edges tables), Concept Graph Engine algorithm, API design & code review
QA / Tester & Fullstack Dev	Nguyễn Minh Phát	Requirements elicitation (SRS), use case model & specifications, test plan & test cases, functional testing, backend development support
Frontend Leader	Nguyễn Phương Gia Bảo	React/Vite app scaffold & routing, UI component library, authentication UI, Dashboard layout, concept graph visualization (react-flow), API integration
Backend Leader	Ngô Văn Phong	Express.js REST API implementation, authentication system (JWT + bcrypt), database models & migrations, Swagger/OpenAPI documentation, mock API responses

Communication channels:

- Discord - primary day-to-day communication (separate channels per role/sprint)
- JIRA - task tracking and sprint management
- GitHub - source code, pull requests, code review
- Google Drive - document storage and sharing

- Weekly Scrum Meeting - every Saturday, 30–60 minutes, full team

3.2 Roles and Responsibilities

RACI Matrix for Key Activities:

Activity	Kiệt (PM)	Quân (Arch)	Phát (QA)	Bảo (FE)	Phong (BE)
Sprint planning & tracking	R/A	C	I	I	I
Architecture design	C	R/A	I	C	C
UI/UX design (wireframes, mockups)	R/A	C	C	C	C
Concept Graph Engine design & impl.	C	R/A	C	I	C
Graph visualization (Dashboard)	I	C	I	R/A	C
Frontend development	I	C	I	R/A	I
Backend API development	I	C	I	I	R/A
AI integration (Gemini API)	I	R	I	C	A
Requirements & use case documentation	C	C	R/A	I	I
Testing	I	I	R/A	C	C
Reporting & documentation	R/A	C	C	I	I

R = Responsible, A = Accountable, C = Consulted, I = Informed

4. Management Process

4.1 Project Estimates

The project has a fixed schedule of 10 weeks across 5 two-week sprints. The team consists of 5 members with zero budget. All tooling uses free tiers. No re-estimation is expected unless a sprint rolls over more than 20% of its tasks, in which case the Project Manager will convene a planning session to adjust the next sprint's scope.

4.2 Project Plan

4.2.1 Phase and Iteration Plan

The project follows the Rational Unified Process (RUP) lifecycle, divided into 5 two-week sprints, with each sprint mapping to a RUP phase.

Sprint	RUP Phase	Timeline	PA	Main Objective
Sprint 1	Inception	Jun 21 – Jun 28	PA0	Team formation, project ideation, draft Project Proposal (PA0), and configure development and management tools.

Sprint 2	Elaboration	Jun 29 – Jul 12	PA1, PA2	Complete PA1 and PA2, define project requirements (Vision, Use Cases), and develop UI prototypes for core screens.
Sprint 3	Elaboration → Construction	Jul 13 – Jul 26	PA3	- Establish baseline codebase (Frontend: React/Vite, Backend: Node/Express), configure database (PostgreSQL/Prisma) and basic CI/CD, and complete detailed system design and Test Plan (PA3). - Implement user authentication flows, develop document upload functionality, integrate Gemini API for concept extraction (AI Study Planner), and build the initial version of the Concept Graph Engine using mock data.
Sprint 4	Construction	Jul 27 – Aug 09	PA4	Finalize PA4. Implement logic for the Focus Session (Pomodoro timer) and the core feature: AI Examiner (multi-turn interview). Integrate grading results with the Concept Graph Engine.
Sprint 5	Construction → Transition	Aug 10 – Aug 23	PA5	Finalize PA5. Visualize the Concept Graph on the Dashboard using React Flow, finalize trace-back functionality (agentic behavior), conduct full system testing, and prepare for the final demo.

Sprint 1 - Inception (Jun 21 – Jun 28) - COMPLETED

Goal: Formed the team, selected the project idea, drafted the Project Proposal (PA0), and set up collaboration tools (GitHub, Jira, Slack).

Iteration Objective	Status
Finalize project proposal (Recall AI concept, scope, tech stack)	Done
Set up team tools: GitHub repo, JIRA board, Discord server, Figma workspace	Done
Conduct ≥ 5 user interviews (UX research)	Done
Evaluate and document tech stack decision	Done
Draw initial system architecture sketch	Done
Submit PA0 to Moodle	Done

Completion rate: 80% (2 tasks rolled over to Sprint 2: Auth tech stack research, Gemini API research).

Sprint 2 - Elaboration (Jun 29 – Jul 12) - IN PROGRESS

Goal: Produce all requirements (Vision, Use Cases) and design artifacts for PA1 + PA2.

ID	Name Task	Description	Objective	Acceptance criteria	Blocki ng	Blocked by	Assignee
----	-----------	-------------	-----------	---------------------	-----------	------------	----------

1	Write Project Plan (Draft)	Open the Project Plan template provided by the TAs. Briefly write the following sections: - 1. Introduction - 2. Project Overview (Problem, goals) - 3. Project Organization (Team roles) - 4.2 Project Plan - 4.3 Project Monitoring (4.3.1, 4.3.3)	Establish a draft of the project plan for PA1 submission, helping TAs understand the team's expected progress.	- Standard English text (use a grammar checker). - All green italicized placeholder text from the template must be removed. - All phases and sprints must match the timeline (5 Sprints).	#5	None	Kiệt
2	Write Vision Document (Draft)	Open the Vision Document template. Fill in the information for: - Team, Problem Statement (from Proposal). - Target Users, Product Features (from Section 5 of the Proposal). - Non-functional requirements (Security, Performance).	Define the product vision to serve as the foundation for Use Cases in PA2.	- Standard English text. - Fully complete the basic required sections in the template. - Quan (Architect) has reviewed the Non-functional requirements section.	#5, #6	None	Kiet, Quan
3	UI Prototyping Design (3-6 screens)	Use Figma (or AI tools) to design 3-6 core screens. The following are mandatory: 1. Landing Page 2. Register/Login 3. Dashboard (displaying basic graphs) 4. Create Study Plan (document upload) 5. AI Examiner (chat interview interface)	Help the Frontend team visualize the UI for development and provide deliverables/proof for PA1.	- Exportable to image files or a public Figma link. - The UI has a logical flow and reflects the core features.	None	None	Kiet, Bao
4	Write	Fill out 2 Weekly	Report	- Completed 2	#8	None	Kiet

	Weekly Report (Sprint 1 - Week 1 & 2)	Report templates for the 2 weeks of the Inception phase (Sprint 1). The content should revolve around brainstorming ideas, writing PA0, and setting up tools.	weekly progress to the TAs.	Word files exactly following the template.			
5	Revise Project Plan & Vision Document	Based on PA1 feedback (or internal review to add more details). Expand on: - Alternatives and competitors (extracted from the Quizlet and NotebookLM comparison table in the Proposal). - User environments. - Revision History (mark modified sections in blue).	Finalize documents according to PA2 standards.	- Modified text must be highlighted in blue (Blue font). - Revision History table is updated.	None	#1, #2	Kiet
6	Draw Use-case Model Diagram	Use StarUML or Visio to draw the Use Case Diagram for the system. - Actors: Student (User), External System (Gemini API). - Core Use Cases: Sign In/Up, Create Study Plan, View Dashboard, Start Focus Session, Take Interview, View History.	Provide an overview of system interactions to serve as the foundation for the Backend architecture.	- Clear and high-resolution exported image file (PNG/JPEG). - Strictly follows UML standards (correct notation for actors, use cases, include, and extend relationships).	#7	#2	Quan, Phong
7	Write Use-case Specification	Use the Use-case Specs & rup_ucspect.docx template to write details for each use case from Task 6. - Each Use Case must clearly detail: Name, Brief Description, Basic Flow (Step 1, 2, 3...), Alternative	Provide a foundation for QA to write Test Cases (PA3) and for Developers to follow flow description	- All main Use Cases must include a Main Flow (Basic Flow). - Core Use Cases (Create Study Plan, Take Interview) must have clear Alternative	None	#6	Phat, Kiet

		Flows (API Errors, Invalid format...), Preconditions, and Postconditions.	s during coding.	Flows. - Use conversational/procedural language (e.g., "The Actor enters X; The System validates X").			
8	Write Weekly Report (Sprint 2)	Fill out the Weekly Report template for Sprint 2.	Report weekly progress.	- Completed exactly following the template.	None	#4	Kiet

Sprint 3 - Elaboration/Construction (Jul 13 – Jul 26)

Goal:

- Set up the baseline code structure for Frontend (React/Vite) and Backend (Node/Express). Configure the database (PostgreSQL + Prisma) and basic CI/CD pipelines (if applicable). Create the detailed system design and Test Plan (PA3).
- Implement the Authentication flow (Sign-In/Sign-Up). Develop the Document Upload feature and integrate the Gemini API for concept extraction (AI Study Planner). Build the first iteration of the Concept Graph Engine using mock data.

Sprint 4 - Construction (Jul 27 – Aug 09)

Goal: Finalize PA4. Code the core logic for the Focus Session (Pomodoro) and the primary feature: AI Examiner (Multi-turn interview). Integrate the grading results into the Concept Graph Engine.

Sprint 5 - Construction/Transition (Aug 10 – Aug 23)

Goal: Finalize PA5. Render the Concept Graph using **react-flow** on the Dashboard. Complete the trace-back feature (Agentic behavior). Conduct end-to-end testing of the entire system and prepare for the Demo.

4.2.2 Releases

Release	Type	Date	Description
v0.1 - Alpha	Internal Demo	Jul 26, 2026 (PA3)	First working build: Auth + AI Study Planner + Concept Graph Engine (mock data) + Focus Session. Deployed to staging. Not ready for real users.
v0.2 - Beta	Internal Demo	Aug 09, 2026 (PA4)	AI Examiner integrated, trace-back algorithm connected to real grading data. Demo-ready for core loop: Plan → Focus → Verify → Trace-back.

v1.0 - MVP	Final Demo	Aug 23, 2026 (PA5)	Full feature set: Dashboard visualization, agentic reminders, session history. System tested end-to-end. Ready for course demo.
------------	------------	--------------------	---

4.2.3 Project Schedule

Key Milestones:

Date	Milestone	Achievement Criteria
Jun 28, 2026	PA0 Submission	Project proposal + team setup complete
Jul 01, 2026	User interviews complete	≥ 5 interviews documented with evidence links
Jul 08, 2026	Use Case specifications done	8 UCs in RUP format, UC diagram in Draw.io
Jul 12, 2026	PA1 + PA2 Submission	Vision doc, SDP, Use Cases, wireframes submitted to Moodle
Jul 26, 2026	PA3 - Alpha release	Auth + Planner + Concept Graph Engine (mock) + Focus Session deployed to staging
Aug 09, 2026	PA4 - Beta release	AI Examiner + trace-back connected to real grading data, demo-ready
Aug 23, 2026	PA5 - Final Submission	Full MVP deployed, demo recorded/performed, all documentation complete

4.2.4 Project Resourcing

Team allocation per sprint:

Member	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
Kiệt (PM+UX)	UX research, proposal	UX research, mockups, SDP	UI polish, user testing	UI testing	Final demo
Quân (Arch)	Architecture sketch	SAD, DB schema	Concept Graph Engine	Trace-back integration	Dashboard graph viz
Phát (QA)	Interview evidence	SRS, Use Cases	Test plan, backend dev	AI Examiner testing	System testing
Bảo (FE)	Frontend research	React scaffold, Auth UI	Core UI screens	AI Examiner UI	Dashboard viz
Phong (BE)	DB research	Auth API, Swagger	Study Planner API	Grading API	Reminder engine

Tools and Infrastructure (all free tier):

Resource	Tool / Platform	Cost
Source control	GitHub	Free
Project management	JIRA (student plan)	Free
Design	Figma (student plan)	Free

Frontend hosting	Vercel	Free tier
Backend hosting	Render	Free tier
Database hosting	Render PostgreSQL	Free tier
AI service	Gemini API (Google)	Free tier
Communication	Discord	Free
Documents	Google Drive	Free

4.3 Project Monitoring and Control

4.3.1 Reporting

Weekly Status Reports:

- Published every Saturday at the end of each sprint week.
- Format: tasks completed this week, blockers encountered, tasks planned for next week, current sprint completion percentage.
- Stored in: /docs/management/weekly-reports/week-N.md on GitHub.

Sprint Review (end of each sprint):

- Full team retrospective meeting (30–60 min, recorded in Discord).
- Sprint review document: what was achieved, what was not, root causes, adjustments for next sprint.

JIRA Board:

- Tasks created for every deliverable before the sprint begins.
- Status columns: To Do → In Progress → In Review → Done.
- PM (Kiệt) reviews JIRA board every Monday and Friday.
- Burndown chart generated automatically to track sprint velocity.

Informal Communication:

- Daily check-ins in Discord #daily-standup channel (async text format: Done / Doing / Blocked).
- Blocker escalation: if a task is blocked for > 1 day, tag PM in #blockers channel.

4.3.2 Risk Management

Risks are scored on a 1–5 scale for both Probability and Impact. Risk Exposure = Probability × Impact.

Risk ID	Risk Description	Prob.	Impact	Exposure	Priority	Mitigation / Contingency Plan
R01	Gemini API rate limit exceeded during demo	3	5	15	High	Cache AI results; limit requests/user/day; implement fallback to static flashcard mode with pre-generated questions

R02	AI returns incorrect prerequisite relationships → poor trace-back quality	4	4	16	High	Mandatory human-in-the-loop confirmation (UC-03) before any study session; UI allows easy node/edge editing
R03	Trace-back algorithm produces a cyclic graph	3	4	12	High	DAG check (cycle detection) runs on every edge save; edges causing cycles are rejected with user notification and logged
R04	Workload imbalance - PM overloaded, Backend tasks not started	4	3	12	High	Explicit RACI per sprint; rollover tasks assigned with hard deadlines; PM task count reduced Sprint 2 (10→6)
R05	Concept Graph Engine not complete by Sprint 3 deadline	2	5	10	Medium	Algorithm unit-tested with mock data independently of AI; minimum demo criterion = graph saves + BFS runs on mock; Dashboard viz deferred to Sprint 5 if needed
R06	AI grading is inconsistent across turns	3	3	9	Medium	Grading rubric extracted from material once at Ingest; AI only grades against that fixed rubric per turn; users can dispute grading results
R07	Team member unavailable mid-sprint	2	4	8	Medium	Each technical component has a primary owner and one secondary (see RACI); critical path items have Quân as backup
R08	Study material with no layered concept structure (sparse graph)	3	2	6	Low	System applies ordinary spaced repetition for isolated concepts; trace-back gracefully skips concepts with no prerequisites
R09	Frontend–Backend interface mismatch	2	3	6	Low	Swagger/OpenAPI documentation due Aug 10; frontend uses mock data until backend API is stable; contract-first API design
R10	Gemini API pricing changes / free tier removed	1	5	5	Low	AI service abstraction layer designed for provider switching; alternative: OpenAI API with identical JSON schema

4.3.3 Configuration Management

Source Code Management:

- Platform: GitHub - repository: github.com/CAGT-Group07/recall-ai
- Branch strategy:
 - main - stable, production-ready code only. Protected branch, requires PR + 1 reviewer approval.
 - dev - integration branch for sprint work.
 - feature/<task-id>-<short-name> - individual feature branches (e.g., feature/S3-UC04-focus-session).
 - fix/<issue-id> - bug fix branches.

- Commit message format: [TASK-ID] Short imperative description (e.g., [S2-22] Add JWT authentication middleware)

Document Management:

- Platform: Google Drive - folder: CAGT Group 07 / Recall AI
- /docs/requirements/ - Vision doc, SRS, personas, user stories, interview findings
- /docs/analysis-and-design/ - SAD, DB schema, wireframes, mockups, use case specs
- /docs/management/ - SDP, weekly reports, sprint reviews
- /pa/pa0/, /pa/pa1/, /pa/pa2/, ... - PA submission packages
- File naming with version number (e.g., SDP_v1.1_08Jul2026.docx)
- All final submission packages are zipped as PA[N]-Group07.zip and uploaded to Moodle.

Code Quality Tools:

- ESLint + Prettier - configured at project init for both frontend and backend.
- TypeScript - strict mode enabled; types/interfaces shared between frontend and backend layers.
- Zod - runtime validation for all API request/response payloads.
- Jest - unit tests for the trace-back algorithm and scheduling logic (no AI dependency required).

Environment Management:

- .env.example committed to repo; actual .env files gitignored.
- Environment variables: DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, GEMINI_API_KEY, NODE_ENV.
- Three environments: development (local), staging (Vercel/Render preview), production (Vercel/Render main).